Generative AI HW2 電機四 4111064113 白舜文

I . Problem Description

1. Modify the example of U-Net for image segmentation of Oxford-IIIT Pet Dataset in the following two aspects. First, reduce the encoder-decoder structure of the neural network to a 2-layer structure with a middle bottleneck layer. Second, change one of the convolutions in the _block to grouped convolution. Train and test the performance of the new network. Please hand in the printed source code and the test results of your work. (Note: If you don't have sufficient computational power, this network will take a long time to train. Please start training as soon as possible. You may also try Google's colab platform: https://colab.research.google.com if you don't have a GPU.)

2-layer structure with a middle bottleneck layer:

```
class UNet(nn.Module):
    def __init__(self, in_channels=3, out_channels=3, init_features=64):
        super(UNet, self).__init__()

        features = init_features
        self.encoder1 = UNet._block(in_channels, features, name="enc1")
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.encoder2 = UNet._block(features, features * 2, name="enc2")
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.bottleneck = UNet._block(features * 2, features * 4, name="bottleneck")

        self.upconv2 = nn.ConvTranspose2d(features * 4, features * 2, kernel_size=2, stride=2)
        self.decoder2 = UNet._block((features * 2) * 2, features * 2, name="dec2")
        self.upconv1 = nn.ConvTranspose2d(features * 2, features, kernel_size=2, stride=2)
        self.decoder1 = UNet._block(features * 2, features, name="dec1")

        self.conv = nn.Conv2d(in_channels=features, out_channels=out_channels, kernel_size=1)

    def forward(self, x):
        enc1 = self.encoder1(x)
        enc2 = self.encoder2(self.pool1(enc1))

        bottleneck = self.bottleneck(self.pool2(enc2))

        dec2 = self.upconv2(bottleneck)
        dec2 = torch.cat((dec2, enc2), dim=1)
        dec2 = self.decoder2(dec2)
        dec1 = self.upconv1(dec2)
        dec1 = torch.cat((dec1, enc1), dim=1)
        dec1 = self.decoder1(dec1)
        return torch.sigmoid(self.conv(dec1))
```

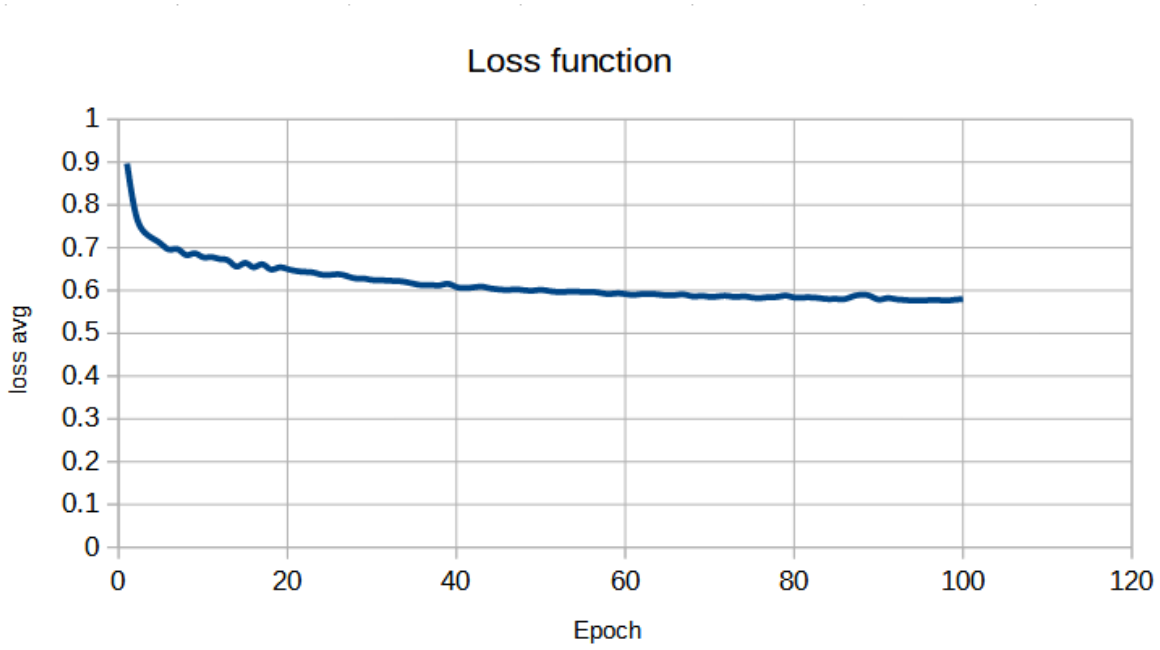
change one of the convolutions in the _block to grouped convolution:

```
def _block(in_channels, features, name):
    group_val = 8 if in_channels % 8 == 0 else 1
    return nn.Sequential(
```

```
    OrderedDict(
      [
        (
          name + "conv1",
          nn.Conv2d(
            in_channels=in_channels,
            out_channels=features,
            kernel_size=3,
            padding=1,
            bias=False,
            groups=group_val
          ),
        ),
        (name + "norm1", nn.BatchNorm2d(num_features=features)),
        (name + "relu1", nn.ReLU(inplace=True)),
        (
          name + "conv2",
          nn.Conv2d(
            in_channels=features,
            out_channels=features,
            kernel_size=3,
            padding=1,
            bias=False,
          ),
        ),
        (name + "norm2", nn.BatchNorm2d(num_features=features)),
        (name + "relu2", nn.ReLU(inplace=True)),
      ]
    )
  )
```

II. Training result

```
criterion = nn.CrossEntropyLoss(reduction='mean')
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```



Environment: C:\AI\venv\Scripts\python.exe C:\AI\unet+segnet\unet.py

Epoch: 01, Batch 0, Loss: 1.0682 loss average: 0.8969
Batch 10, Loss: 0.9419
Batch 20, Loss: 0.8881
Batch 30, Loss: 0.8563
Batch 40, Loss: 0.8152
Batch 50, Loss: 0.8116

Epoch: 10, Batch 0, Loss: 0.6688 loss average: 0.6779
Batch 10, Loss: 0.6852
Batch 20, Loss: 0.6793
Batch 30, Loss: 0.6745
Batch 40, Loss: 0.6722
Batch 50, Loss: 0.6871

Epoch: 20, Batch 0, Loss: 0.6444 loss average: 0.6502
Batch 10, Loss: 0.6543
Batch 20, Loss: 0.6508
Batch 30, Loss: 0.6589
Batch 40, Loss: 0.6361
Batch 50, Loss: 0.6569

Epoch: 30, Batch 0, Loss: 0.6262 loss average: 0.6247
Batch 10, Loss: 0.6265
Batch 20, Loss: 0.6183
Batch 30, Loss: 0.6218
Batch 40, Loss: 0.6210
Batch 50, Loss: 0.6345

Epoch: 40, Batch 0, Loss: 0.6121 loss average: 0.6082
Batch 10, Loss: 0.6105
Batch 20, Loss: 0.6068
Batch 30, Loss: 0.6050
Batch 40, Loss: 0.6058
Batch 50, Loss: 0.6092

Epoch: 50, Batch 0, Loss: 0.6013 loss average: 0.6013
Batch 10, Loss: 0.6072
Batch 20, Loss: 0.5960
Batch 30, Loss: 0.6018

Batch 40, Loss: 0.5994
Batch 50, Loss: 0.6021
Epoch: 60, Batch 0, Loss: 0.5911 loss average: 0.5918
Batch 10, Loss: 0.5910
Batch 20, Loss: 0.5963
Batch 30, Loss: 0.5909
Batch 40, Loss: 0.5891
Batch 50, Loss: 0.5923
Epoch: 70, Batch 0, Loss: 0.5815 loss average: 0.5854
Batch 10, Loss: 0.5837
Batch 20, Loss: 0.5871
Batch 30, Loss: 0.5865
Batch 40, Loss: 0.5890
Batch 50, Loss: 0.5843
Epoch: 80, Batch 0, Loss: 0.5837 loss average: 0.5838
Batch 10, Loss: 0.5849
Batch 20, Loss: 0.5841
Batch 30, Loss: 0.5820
Batch 40, Loss: 0.5844
Batch 50, Loss: 0.5835
Epoch: 90, Batch 0, Loss: 0.5802 loss average: 0.5790
Batch 10, Loss: 0.5788
Batch 20, Loss: 0.5793
Batch 30, Loss: 0.5805
Batch 40, Loss: 0.5787
Batch 50, Loss: 0.5765
Epoch: 100, Batch 0, Loss: 0.5763 loss average: 0.5798
Batch 10, Loss: 0.5762
Batch 20, Loss: 0.5791
Batch 30, Loss: 0.5809
Batch 40, Loss: 0.5838
Batch 50, Loss: 0.5826

During training, I found that there’re no plt.figure() functions inside like HW1, so I used excel to draw the loss function figure instead.

III. Testing for two results

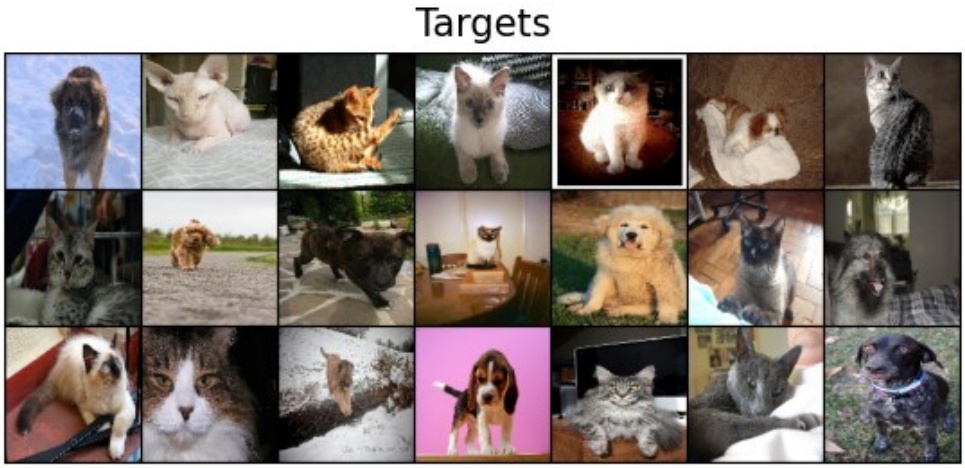
Environment: C:\AI\venv\Scripts\python.exe C:\AI\unet+segnet\unet_test.py

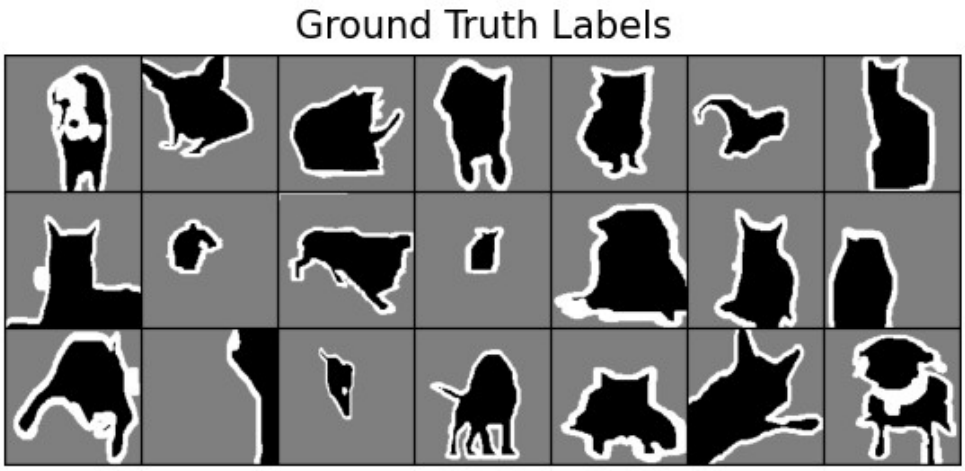
device: cuda

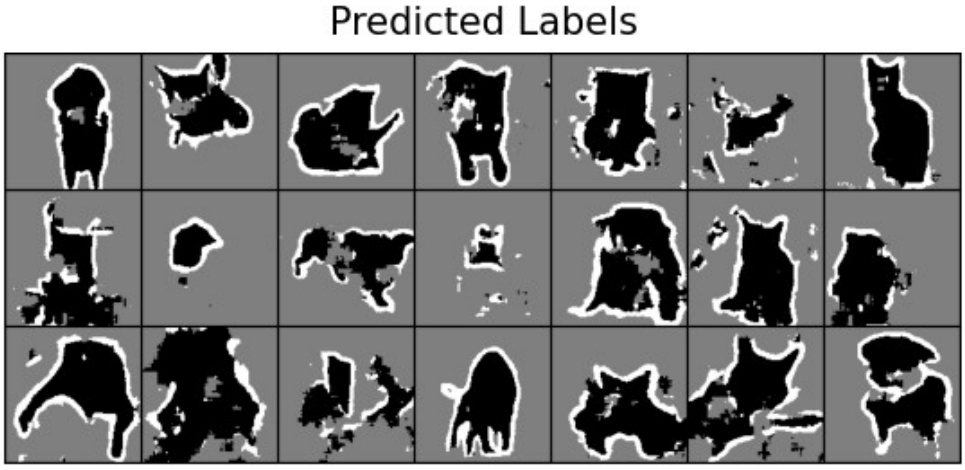
100.0%

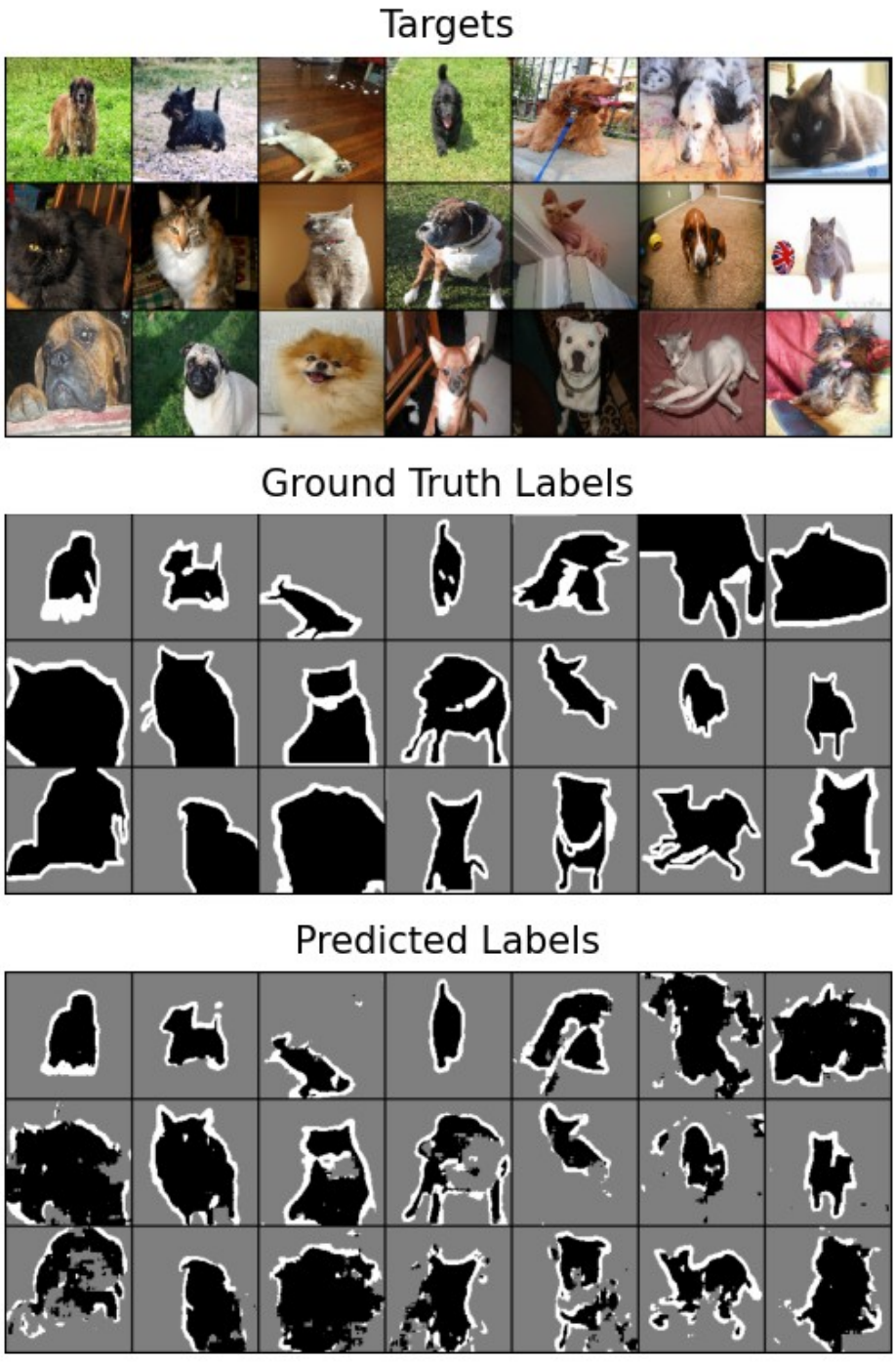
100.0%

The Model has 1.22M parameters









IV. Source code

1. unet.py:

```
import torch
from torch import nn
import torchvision
import torchvision.transforms as T
from collections import OrderedDict

# https://github.com/mateuszbuda/brain-segmentation-pytorch

def tensor_trimap(t):
    x = t * 255
```

```
    x = x.to(torch.long)
    x = x - 1
    return x

class UNet(nn.Module):
    def __init__(self, in_channels=3, out_channels=3, init_features=64):
        super(UNet, self).__init__()

        features = init_features
        self.encoder1 = UNet._block(in_channels, features, name="enc1")
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.encoder2 = UNet._block(features, features * 2, name="enc2")
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
        # self.encoder3 = UNet._block(features * 2, features * 4, name="enc3")
        # self.pool3 = nn.MaxPool2d(kernel_size=2, stride=2)

        # self.bottleneck = UNet._block(features * 4, features * 8, name="bottleneck")
        self.bottleneck = UNet._block(features * 2, features * 4, name="bottleneck")

        self.upconv2 = nn.ConvTranspose2d(
            features * 4, features * 2, kernel_size=2, stride=2
        )
        self.decoder2 = UNet._block((features * 2) * 2, features * 2, name="dec2")
        self.upconv1 = nn.ConvTranspose2d(
            features * 2, features, kernel_size=2, stride=2
        )
        self.decoder1 = UNet._block(features * 2, features, name="dec1")
        # self.upconv1 = nn.ConvTranspose2d(
            #features * 2, features, kernel_size=2, stride=2
        #)
        #self.decoder1 = UNet._block(features * 2, features, name="dec1")

        self.conv = nn.Conv2d(
            in_channels=features, out_channels=out_channels, kernel_size=1
        )

    def forward(self, x):
        enc1 = self.encoder1(x)
        enc2 = self.encoder2(self.pool1(enc1))
        # enc3 = self.encoder3(self.pool2(enc2))

        bottleneck = self.bottleneck(self.pool2(enc2))

        # dec3 = self.upconv3(bottleneck)
        # dec3 = torch.cat((dec3, enc3), dim=1)
        # dec3 = self.decoder3(dec3)
        dec2 = self.upconv2(bottleneck)
        dec2 = torch.cat((dec2, enc2), dim=1)
        dec2 = self.decoder2(dec2)
        dec1 = self.upconv1(dec2)
        dec1 = torch.cat((dec1, enc1), dim=1)
        dec1 = self.decoder1(dec1)
        return torch.sigmoid(self.conv(dec1))

    def _block(in_channels, features, name):
        group_val = 8 if in_channels % 8 == 0 else 1
        return nn.Sequential(
            OrderedDict(
                [
                    (
                        name + "conv1",
                        nn.Conv2d(
                            in_channels=in_channels,
```

```
                        out_channels=features,
                        kernel_size=3,
                        padding=1,
                        bias=False,
                        groups=group_val
                    ),
                ),
                (name + "norm1", nn.BatchNorm2d(num_features=features)),
                (name + "relu1", nn.ReLU(inplace=True)),
                (
                    name + "conv2",
                    nn.Conv2d(
                        in_channels=features,
                        out_channels=features,
                        kernel_size=3,
                        padding=1,
                        bias=False,
                    ),
                ),
                (name + "norm2", nn.BatchNorm2d(num_features=features)),
                (name + "relu2", nn.ReLU(inplace=True)),
            ]
        )
    )

def train_loop(model, loader, epochs, device):
    criterion = nn.CrossEntropyLoss(reduction='mean')
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

    epoch_i, epoch_j = epochs
    for i in range(epoch_i, epoch_j):
        # print(f"Epoch: {i:02d}, Learning Rate: {optimizer.param_groups[0]['lr']}")
        print(f"Epoch: {i:02d}")
        running_loss = 0.0
        # running_samples = 0
        for batch_idx, (inputs, targets) in enumerate(loader, 0):
            inputs = inputs.to(device)
            targets = targets.to(device)

            optimizer.zero_grad()
            outputs = model(inputs)

            # targets = targets.squeeze(dim=1)
            # loss = criterion(outputs, targets)
            loss = criterion(outputs, targets.squeeze(dim=1))
            loss.backward()
            optimizer.step()

            # running_samples += targets.size(0)
            running_loss += loss.item()
            if batch_idx % 10 == 0:
                print(f"Batch {batch_idx}, Loss: {loss.item():.4f}")
            # print("Trained {} samples, Loss: {:.4f}".format(running_samples, running_loss / (batch_idx+1)))

if torch.cuda.is_available():
  device = torch.device("cuda")
else:
  device = torch.device("cpu")
print('device:', device)

transforms = T.Compose([T.ToTensor(), T.Resize((128, 128), interpolation=T.InterpolationMode.NEAREST)])
target_transforms = T.Compose([T.ToTensor(), T.Resize((128, 128), interpolation=T.InterpolationMode.NEAREST),
T.Lambda(tensor_trimap)])
```

```
pets_train = torchvision.datasets.OxfordIIITPet(root="datasets/OxfordPets/train/", split="trainval",
target_types="segmentation",
                    download=True, transform=transforms, target_transform=target_transforms)
pets_train_loader = torch.utils.data.DataLoader(pets_train, batch_size=64, shuffle=True)

model = UNet()
model = model.to(device)
model.train()

train_loop(model, pets_train_loader, (1, 101), device)

torch.save(model.state_dict(), 'unet.pth')
```

2. unet_test.py:

```
import torch
from torch import nn
import torchvision
import torchvision.transforms as T
from collections import OrderedDict
from matplotlib import pyplot as plt

t2img = T.ToPILImage()

def tensor_trimap(t):
    x = t * 255
    x = x.to(torch.long)
    x = x - 1
    return x

class UNet(nn.Module):
    def __init__(self, in_channels=3, out_channels=3, init_features=64):
        super(UNet, self).__init__()

        features = init_features
        self.encoder1 = UNet._block(in_channels, features, name="enc1")
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.encoder2 = UNet._block(features, features * 2, name="enc2")
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
        # self.encoder3 = UNet._block(features * 2, features * 4, name="enc3")
        # self.pool3 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.bottleneck = UNet._block(features * 2, features * 4, name="bottleneck")

        self.upconv2 = nn.ConvTranspose2d(
            features * 4, features * 2, kernel_size=2, stride=2
        )
        self.decoder2 = UNet._block(features * 4, features * 2, name="dec2")
        self.upconv1 = nn.ConvTranspose2d(
            features * 2, features, kernel_size=2, stride=2
        )
        self.decoder1 = UNet._block(features * 2, features, name="dec1")
        # self.upconv1 = nn.ConvTranspose2d(
            # features * 2, features, kernel_size=2, stride=2
        # )
        # self.decoder1 = UNet._block(features * 2, features, name="dec1")

        self.conv = nn.Conv2d(
            in_channels=features, out_channels=out_channels, kernel_size=1
        )

    def forward(self, x):
```

```
        enc1 = self.encoder1(x)
        enc2 = self.encoder2(self.pool1(enc1))
        # enc3 = self.encoder3(self.pool2(enc2))

        bottleneck = self.bottleneck(self.pool2(enc2))

        # dec3 = self.upconv3(bottleneck)
        # dec3 = torch.cat((dec3, enc3), dim=1)
        # dec3 = self.decoder3(dec3)
        dec2 = self.upconv2(bottleneck)
        dec2 = torch.cat((dec2, enc2), dim=1)
        dec2 = self.decoder2(dec2)
        dec1 = self.upconv1(dec2)
        dec1 = torch.cat((dec1, enc1), dim=1)
        dec1 = self.decoder1(dec1)
        return torch.sigmoid(self.conv(dec1))

    def _block(in_channels, features, name):
        group_val = 8 if in_channels % 8 == 0 else 1
        return nn.Sequential(
            OrderedDict(
                [
                    (
                        name + "conv1",
                        nn.Conv2d(
                            in_channels=in_channels,
                            out_channels=features,
                            kernel_size=3,
                            padding=1,
                            bias=False,
                            groups=group_val
                        ),
                    ),
                    (name + "norm1", nn.BatchNorm2d(num_features=features)),
                    (name + "relu1", nn.ReLU(inplace=True)),
                    (
                        name + "conv2",
                        nn.Conv2d(
                            in_channels=features,
                            out_channels=features,
                            kernel_size=3,
                            padding=1,
                            bias=False,
                        ),
                    ),
                    (name + "norm2", nn.BatchNorm2d(num_features=features)),
                    (name + "relu2", nn.ReLU(inplace=True)),
                ]
            )
        )

if torch.cuda.is_available():
  device = torch.device("cuda")
else:
  device = torch.device("cpu")
print('device:', device)

transforms = T.Compose([T.ToTensor(), T.Resize((128, 128), interpolation=T.InterpolationMode.NEAREST)])
target_transforms = T.Compose([T.ToTensor(), T.Resize((128, 128), interpolation=T.InterpolationMode.NEAREST),
T.Lambda(tensor_trimap)])
pets_test = torchvision.datasets.OxfordIIITPet(root="datasets/OxfordPets/test/", split="test",
target_types="segmentation",
                       download=True, transform=transforms, target_transform=target_transforms)
```

```
pets_test_loader = torch.utils.data.DataLoader(pets_test, batch_size=21, shuffle=True)

model = UNet()
model.load_state_dict(torch.load('unet.pth',weights_only=True))
model = model.to(device)
model.eval()

with torch.inference_mode():
    total_params = sum(param.numel() for param in model.parameters())
    print(f"The Model has {total_params/1e6:.2f}M parameters")

    (inputs, targets) = next(iter(pets_test_loader))
    inputs = inputs.to(device)
    targets = targets.to(device)
    predictions = model(inputs)
    # pred = nn.Softmax(dim=1)(predictions)
    pred_labels = predictions.argmax(dim=1)
    pred_labels = pred_labels.unsqueeze(1)
    pred_mask = pred_labels.to(torch.float)

    fig = plt.figure(figsize=(10, 12))
    fig.add_subplot(3, 1, 1)
    plt.imshow(t2img(torchvision.utils.make_grid(inputs, nrow=7)))
    plt.axis('off')
    plt.title("Targets")

    fig.add_subplot(3, 1, 2)
    plt.imshow(t2img(torchvision.utils.make_grid(targets.float() / 2.0, nrow=7)))
    plt.axis('off')
    plt.title("Ground Truth Labels")

    fig.add_subplot(3, 1, 3)
    plt.imshow(t2img(torchvision.utils.make_grid(pred_mask / 2.0, nrow=7)))
    plt.axis('off')
    plt.title("Predicted Labels")

    plt.show()
```